

ASSAR Pricing Engine: Calculator Walkthroughs

A step-by-step guide to every function in `assar/pricing/`. Each calculator is shown as the sequence of steps it performs, with the key code and a plain explanation of each line. The aim is that the whole engine can be understood and maintained end to end.

Two invariants hold throughout:

- No answer is hard-coded. Every rate is read from SQLite via `get\_rate()` or `\_transit\_rate()`; the functions only compose those exact rates.
- Every calculator returns a `Quote` object (defined in `base.py`) whose `lines` list is the human-readable breakdown the UI shows.

A calculator typically follows the same shape:

```
own = conn is None          # open a DB connection if the caller didn't pass one
conn = conn or connect()
try:
    q = Quote(product=..., sum_insured=...)
    rate, unit = get_rate(scheme, category, conn=conn) # exact rate from SQLite
    q.gross_premium = premium_from_rate(sum_insured, rate, unit)
    ... apply loadings / discounts / multipliers / short period ...
    ... floor at minimum premium and/or add the policy fee ...
    q.final_premium = ...
    return q
finally:
    if own:
        conn.close()          # only close what we opened
```

The `own`/`finally` pattern lets calculators share one connection when called together, while still working standalone. The walkthroughs below omit that boilerplate and focus on the pricing logic.

0. Shared building blocks (base.py)

`get_rate(scheme, category, alt=False)`

```
col = "rate_alt" if alt else "rate"
row = fetch(category)                # SELECT col, unit WHERE scheme=? AND category=?
if row is None or row["r"] is None:
    snapped = _closest_category(conn, scheme, category) # fuzzy fallback
    if snapped is not None:
        row = fetch(snapped)
if row is None or row["r"] is None:
    raise RateNotFound(...)
return float(row["r"]), row["unit"]
```

1. Choose the column: `rate`, or `rate\_alt` when `alt=True` (the second value a

- table carries, e.g. fire's "all special perils" rate).
2. Try an exact lookup of the category within the scheme.
 3. On a miss, ``_closest_category`` does a scheme-scoped fuzzy match (difflib plus a substring check), so "hotel" resolves to "hotels". This stays within the same scheme, so it can never cross product families.
 4. If still nothing, raise ``RateNotFound`` (surfaced to the caller as an error).
 5. Return the rate as a float plus its unit (``percent``, ``per_mille``, ``amount``).

premium_from_rate(sum_insured, rate, unit="percent")

```
if unit == "per_mille":
    return sum_insured * rate / 1000.0
if unit == "amount":
    return rate
return sum_insured * rate / 100.0
```

The one place the actual multiply happens. Percent divides by 100, per mille by 1000, and an ``amount`` rate (flat figures such as school premiums) is returned as-is.

voluntary_deductible_discount(excess_amount, gross_premium)

```
row = SELECT value FROM schedule WHERE name='voluntary_deductible'
      AND (? >= lower) AND (? < upper) ORDER BY ord DESC LIMIT 1
pct = float(row["value"])
saving = gross_premium * pct / 100.0
cap = excess_amount * 33.33 / 100.0
return pct, min(saving, cap)
```

Finds the discount percent for the band the chosen excess falls in, then caps the premium saving at 33.33 percent of the excess amount (the manual's rule).

short_period_fraction(period_months, period_days=None, schedule="short_period_months")

```
if period_months is None and period_days is None: return 1.0
if period_days is not None and period_days < 28:  # day-scale lookup
    ...; return value
if period_months >= 12: return 1.0
row = SELECT value FROM schedule WHERE name=? AND upper >= ? ORDER BY upper ASC LIMIT 1
return float(row["value"]) if row else 1.0
```

Returns the fraction of the annual premium for short cover (for example three months gives 0.5). A full year, or no period given, returns 1.0. Some classes pass a different ``schedule`` name (school, PA/GPA, CPM) because their short-rate scales differ.

apply_minimum, product_rule, policy_fee

- ``apply_minimum(premium, minimum, quote)``: if the premium is below the class minimum, charge the minimum (and note it in the breakdown).
 - ``product_rule(product, key, default)``: read a per-product constant from the ``product_rule`` table (minimum premiums, excess percentages, loadings).
 - ``policy_fee()``: the Rwf5,000 market policy fee.
-

1. Fire family (fire.py)

**quote_fire(risk_category, sum_insured, special_perils=True,
industrial=False, fea_available=False, voluntary_excess=0,
period_months=None)**

```
base_rate, _ = get_rate("fire", risk_category, alt=special_perils, conn=conn)
fire_portion = premium_from_rate(sum_insured, base_rate)
if industrial:
    load = product_rule("fire", "industrial_load", 0.025)          # +0.025%
    process_portion = premium_from_rate(sum_insured, load)
    ext_load = product_rule("fire", "industrial_ext_load", 25_000)  # flat Rwf25,000
q.gross_premium = fire_portion + process_portion + ext_load
net = q.gross_premium
if fea_available:
    fea = product_rule("fire", "fea_discount", 15.0)
    net -= fire_portion * fea / 100.0          # FEA applies to fire portion only
if voluntary_excess > 0:
    pct, saving = voluntary_deductible_discount(voluntary_excess, net)
    net -= saving
q.net_premium = net * short_period_fraction(period_months)
q.final_premium = q.net_premium + policy_fee()
```

1. Read the occupancy rate. ``alt=special_perils`` selects the all-special-perils column when special perils are wanted, otherwise the standard-fire column.
2. The fire portion is ``sum insured x rate``.
3. Industrial risks add a 0.025 percent process loading plus a flat Rwf25,000 extensions loading; the FEA discount must not apply to these.
4. Gross premium is fire portion plus those loadings.
5. The 15 percent FEA discount is subtracted from the fire portion only.
6. A voluntary excess earns the banded deductible discount (already capped).
7. Apply the short-period factor, add the policy fee, and warn about the Condition of Average.

**quote_consequential_loss(risk_category, gross_profit_si,
indemnity_period_months=12, cover="gross_profit", period_months=None)**

```
fire_rate, _ = get_rate("fire", risk_category)          # the basis rate
key = {"gross_profit": "gross_profit_pct", "auditors_fees": "auditors_fees_pct",
      "wages": "wages_pct"}[cover]
```

```

cover_pct = product_rule("consequential_loss", key, 100.0)      # 150 / 125 / 100
basis_rate = fire_rate * cover_pct / 100.0
ip_mult = SELECT value FROM schedule WHERE name='ci_indemnity' AND upper >= ? # by months
eff_rate = basis_rate * ip_mult / 100.0
q.gross_premium = premium_from_rate(gross_profit_si, eff_rate)
q.net_premium = q.gross_premium * short_period_fraction(period_months)
q.final_premium = q.net_premium + policy_fee()

```

1. The basis is the fire material-damage rate for the same risk.
2. Multiply by the cover factor: gross profit 150, auditors fees 125, wages 100 percent.
3. Multiply by the indemnity-period multiplier looked up from the `ci\_indemnity` schedule by the chosen number of months.
4. That effective rate prices the gross-profit sum insured; short period; fee.
The mandatory time excess is 14 days.

quote_burglary(sum_insured, high_value=False, first_loss_ratio=None, stock_declaration=False, period_months=None)

```

rate = 0.5 if high_value else 0.3
q.gross_premium = premium_from_rate(sum_insured, rate)
net = q.gross_premium
if first_loss_ratio is not None:
    mult = SELECT value FROM schedule WHERE name='first_loss' AND upper >= ? # by ratio
    net = net * mult / 100.0
if stock_declaration:
    net *= (1 - product_rule("burglary", "stock_declaration_discount", 10.0)/100.0)
q.net_premium = net * short_period_fraction(period_months)
q.final_premium = q.net_premium + policy_fee()
q.excess = "10% of each and every loss, min Rwf50,000"

```

1. Full-value rate: 0.3 percent ordinary goods, 0.5 percent high-value.
2. On a first-loss basis, multiply by the multiplier for the first-loss ratio band (from the `first\_loss` schedule).
3. A stock-declaration policy takes a 10 percent discount.
4. Short period; policy fee; mandatory excess 10 percent, min Rwf50,000.

2. Liability, accident, engineering, specialty (products.py)

quote_liability(kind, occupation, limit_of_indemnity, period_months=None)

```

scheme, product = _LIABILITY[kind]      # public/employers/product/professional
rate, _ = get_rate(scheme, occupation)
q.gross_premium = premium_from_rate(limit_of_indemnity, rate)
q.net_premium = q.gross_premium * short_period_fraction(period_months)
minimum = 25_000 if (product=="professional_indemnity" and occupation=="insurance_agents")
            else (200_000 if product=="professional_indemnity" else 100_000)
q.final_premium = apply_minimum(q.net_premium, minimum)
if product == "professional_indemnity":

```

```
q.excess = "5% of each and every loss, min Rwf200,000"
```

1. `kind` selects the scheme and product (public, employers, product, professional).
2. Premium is the limit of indemnity times the occupation rate; short period.
3. Floor at the minimum premium: 100k normally, 200k for professional indemnity, 25k for insurance agents.
4. Professional indemnity carries a 5 percent excess (min Rwf200,000).

```
quote_pa_gpa(risk_class, death_benefit, group=False,  
benefits=("death","tpd"), student=False, period_months=None)
```

```
base, _ = get_rate("pa_gpa", risk_class)  
for b in benefits:  
    if b in ("death","tpd"):      p = premium_from_rate(death_benefit, base)  
    elif b == "ttd":              p = premium_from_rate(death_benefit, base*0.15)  
    elif b in ("medical","funeral"): p = premium_from_rate(death_benefit, base*10)  
    total += p  
q.net_premium = total * short_period_fraction(period_months, schedule="short_period_school")  
minimum = (15_000 if student else 25_000) if not group else (30_000 if student else 50_000)  
q.final_premium = apply_minimum(q.net_premium, minimum)
```

1. One base rate per occupation class.
2. Each selected benefit is priced: death and total permanent disability at the base rate; total temporary disability at 15 percent of it; medical and funeral at ten times it. The benefits are summed against the capital sum.
3. Short period uses the school/PA scale; floor at the PA or GPA minimum (lower for interning students).

```
quote_bond(bond_type, bond_value, cash_collateral_100=False)
```

```
rate, _ = get_rate("bond", bond_type)  
if cash_collateral_100:  
    rate = product_rule("bond", "cash_collateral_rate", 3.0)      # 5% -> 3%  
q.gross_premium = premium_from_rate(bond_value, rate)  
minimum = product_rule("bond", "min_premium_bid" if bond_type=="bid_bond"  
                        else "min_premium_other", ...)             # 10k / 30k  
q.final_premium = apply_minimum(q.gross_premium, minimum)  
q.warn("Bonds carry full rate for any period; no short-period or pro-rata.")
```

1. Bond value times the bond-type rate.
2. Full cash collateral reduces the rate to 3 percent.
3. Floor at Rwf10,000 (bid bond) or Rwf30,000 (other bonds). No short period.

```
quote_pvt(risk_type, sum_insured, security_features_discount=0)
```

```
rate, unit = get_rate("pvt", risk_type)      # unit == 'per_mille'  
q.gross_premium = premium_from_rate(sum_insured, rate, unit=unit)  # /1000  
net = q.gross_premium
```

```

if security_features_discount:
    net *= (1 - min(security_features_discount, 10.0)/100.0)
q.final_premium = net
ded = max(sum_insured * 0.5/100.0, 50_000)
q.excess = f"5% each loss, min 0.5% of SI ({ded})"

```

1. PVT rates are per mille, so the premium divides by 1000, not 100.
2. Approved security features earn up to a 10 percent discount.
3. Mandatory deductible: 5 percent of each loss, minimum 0.5 percent of the sum insured, floored at Rwf50,000.

quote_car_ear(kind, project_type, contract_value, duration_months=12, tpl_limit=0)

```

rate, _ = get_rate("ear_car", project_type)
if duration_months > 12:
    blocks = ceil((duration_months - 12) / 6)
    rate *= (1 + blocks*25.0/100.0) # +25% per extra 6 months
works = premium_from_rate(contract_value, rate)
q.gross_premium = works
cap = product_rule(kind, "tpl_pct_cap", 15.0)
if tpl_limit > 0 and tpl_limit > contract_value*cap/100.0:
    q.gross_premium += premium_from_rate(tpl_limit, seed.EAR_CAR_TPL_SEPARATE) # 0.2%
q.final_premium = q.gross_premium + policy_fee()

```

1. Contract value times the project-type rate (same list for CAR and EAR).
2. Projects beyond twelve months are loaded 25 percent for each extra six-month block (rounded up).
3. Third-party liability is included while within 15 percent of the contract value; above that it is rated separately at 0.2 percent and added.
4. Add the policy fee.

quote_machinery(machine_type, sum_insured, period_months=None)

```

rate, _ = get_rate("machinery", machine_type)
q.gross_premium = premium_from_rate(sum_insured, rate)
q.final_premium = q.gross_premium + policy_fee()
q.excess = ("10% min Rwf500,000" if sum_insured > 5_000_000
            else "5% min Rwf250,000")

```

Sum insured times the machine/industry rate, plus the policy fee. The excess tier depends on whether the sum insured exceeds Rwf5,000,000.

quote_cpm(plant_group, hazard_class, sum_insured, period_months=None)

```

rate, _ = get_rate("cpm", f"{hazard_class}/{plant_group}") # e.g. "B/2"
q.gross_premium = premium_from_rate(sum_insured, rate)
q.net_premium = q.gross_premium * short_period_fraction(period_months,
                                                            schedule="short_period_cpm")
q.final_premium = q.net_premium + policy_fee()
q.excess = "10% of claim, min Rwf500,000"

```

The rate comes from the hazard-class by plant-group matrix (the category is the two joined, e.g. `B/2`). CPM has its own short-period scale. Plus policy fee.

quote_fidelity(risk, sum_insured, blanket=False, employees=0, period_months=None)

```
rate, _ = get_rate("fidelity", risk)
if blanket:
    q.gross_premium = product_rule("fidelity", "blanket_per_capita", 30_000) * employees
else:
    q.gross_premium = premium_from_rate(sum_insured, rate)
q.net_premium = q.gross_premium * short_period_fraction(period_months)
q.final_premium = apply_minimum(q.net_premium, product_rule("fidelity", "min_premium", 200_000))
q.excess = "Rwf250,000 or 10% of adjusted claim, whichever is higher"
```

Either a percent of the guarantee amount, or Rwf30,000 per employee for blanket cover; short period; floor at Rwf200,000; excess the higher of Rwf250,000 or 10 percent.

quote_bbb(limit_of_indemnity) and quote_do_liability(limit_of_indemnity, risk="financial_services")

```
# BBB
rate, _ = get_rate("bbb", "financial_services") # 5%
q.final_premium = q.gross_premium = premium_from_rate(limit_of_indemnity, rate)
# D&O
rate, _ = get_rate("do_liability", risk) # 5% financial / 2.5% other
q.final_premium = q.gross_premium = premium_from_rate(limit_of_indemnity, rate)
```

Both are a single rate on the selected limit, with the excess "Rwf250,000 or 10 percent of adjusted claim, whichever is higher". BBB is financial services only; D&O distinguishes financial services (5 percent) from other offices (2.5).

quote_school_liability(school_category, num_students, period_months=None)

```
per_student, _ = get_rate("school_liability", school_category) # a flat amount
q.gross_premium = per_student * num_students
q.net_premium = q.gross_premium * short_period_fraction(period_months,
                                                         schedule="short_period_school")
q.final_premium = q.net_premium # premiums are annual, incl. fees & VAT
```

The rate is a flat premium per student (unit `amount`), multiplied by the number of students. School premiums are inclusive of fees and VAT, so no separate fee.

quote_aviation(risk_class, sum_insured, seats=1)

```
rate, _ = get_rate("aviation", risk_class)
prem = premium_from_rate(sum_insured, rate)
if "pax" in risk_class:
    prem *= seats # passenger liability is per seat
q.final_premium = prem + policy_fee()
```

A single rate on the hull value or selected limit. For passenger (PAX)

liability, the per-seat figure is multiplied by the number of seats. Plus fee.

quote_marine_hull / quote_boiler / quote_ear / quote_plate_glass

```
# marine hull: hull_all_risks 0.8% of vessel value, or third_party_liability 0.25%
# boiler:      material_damage / third_party_liability 0.5%; excess 10% min 625k
# ear:         premises 0.75% / portable 2% / unspecified 1.5% / increased_cost 0.75%
# plate glass: 2%; excess 5% min 100k
rate, _ = get_rate(scheme, category)
q.gross_premium = premium_from_rate(sum_insured, rate)
q.final_premium = q.gross_premium + policy_fee()
q.excess = product_rule(scheme, ...)      # where the class defines one
```

These four share the simplest shape: one rate on the sum insured, plus the policy fee, with the class excess attached. The `cover`/`location` argument just selects which category (and therefore which rate) to read.

3. Transit (transit.py)

_transit_rate(scheme, commodity, cover, containerized) (helper)

```
col = {("road_accident",True):"ra_containerized",
      ("road_accident",False):"ra_noncontainerized",
      ("all_risks",True):"ar_containerized",
      ("all_risks",False):"ar_noncontainerized"}[(cover, containerized)]
row = fetch(commodity)
if row is None:
    row = fetch(_closest_commodity(scheme, commodity, conn))    # fuzzy fallback
return float(row[col]), row["excess"]
```

Picks the correct cell from the 2x2 grid (road-accident vs all-risks, by whether the cargo is containerized), with a fuzzy commodity fallback. Returns the rate and the excess text.

quote_git(commodity, consignment_value, cover="all_risks", containerized=True, transporters_liability=False, outside_rwanda=False, trips_period_months=None)

```
rate, excess = _transit_rate("git", commodity, cover, containerized, conn)
if transporters_liability and outside_rwanda:
    rate *= 1.30                                     # +30% outside Rwanda
q.gross_premium = premium_from_rate(consignment_value, rate)
q.net_premium = q.gross_premium
if trips_period_months is not None:
    mult = next(m for cap,m in [(3,30),(6,60),(9,90),(12,100)] if trips_period_months<=cap)
    q.net_premium = q.gross_premium * mult/100.0    # multi-trip scaling
q.final_premium = q.net_premium + policy_fee()
```

1. Base rate from the commodity grid, by cover type and packing.
2. Transporters liability outside Rwanda loads the rate by 30 percent.
3. For multiple trips, scale the annual premium by trip period (30/60/90/100

percent). Plus the policy fee.

```
quote_marine_cargo(commodity, consignment_value, containerized=True,  
mode="combined", clause="A")
```

```
rate, excess = _transit_rate("marine_cargo", commodity, "all_risks", containerized, conn)  
rate *= (1 - {"combined":0,"road":10,"air":30,"sea":20}[mode]/100.0) # mode discount  
rate *= (1 - {"A":0,"B":25,"C":35}[clause]/100.0) # clause discount  
q.gross_premium = premium_from_rate(consignment_value, rate)  
q.final_premium = q.gross_premium + policy_fee()
```

1. Base is the Institute Cargo Clause A rate for the commodity and packing.
2. Apply the transit-mode discount first (road 10, sea 20, air 30 percent).
3. Then the clause discount (Clause B 25, Clause C 35 percent). Plus the fee.

4. The dispatcher (registry.py)

```
run_tool(name, args)
```

```
fn = DISPATCH.get(name)  
args = _coerce_numbers(args) # "1000000" -> 1000000  
params = inspect.signature(fn).parameters  
if no **kwargs: args = {k:v for k,v in args.items() if k in params} # drop unknowns  
try:  
    return fn(**args).as_dict()  
except Exception as exc:  
    return {"error": str(exc)}
```

The single entry point the chat uses. It coerces numeric strings to numbers, drops any argument the calculator does not accept (so a stray model-generated keyword cannot crash a quote), calls the calculator, and returns the `Quote` as a dictionary. Errors are returned as data rather than raised, so the chat can recover and the deterministic Get a Quote tab is never affected.

End of walkthroughs.